modelling

MDPI

Article

GPU-Accelerated FLIP Fluid Simulation Based on Spatial Hashing Index and Thread Block-Level Cooperation

Changjun Zou * and Hui Luo

School of Information and Software Engineering, East China Jiaotong University, Nanchang 330013, China; 2024068081200004@ecjtu.edu.cn
* Correspondence: zouchangjun@ecjtu.edu.cn

Abstract

The Fluid Implicit Particle (FLIP) method is widely adopted in fluid simulation due to its computational efficiency and low dissipation. However, its high computational complexity makes it challenging for traditional CPU architectures to meet real-time requirements. To address this limitation, this work migrates the FLIP method to the GPU using the CUDA framework, achieving a transition from conventional CPU computation to large-scale GPU parallel computing. Furthermore, during particle-to-grid (P2G) mapping, the conventional scattering strategy suffers from significant performance bottlenecks due to frequent atomic operations. To overcome this challenge, we propose a GPU parallelization strategy based on spatial hashing indexing and thread block-level cooperation. This approach effectively avoids atomic contention and significantly enhances parallel efficiency. Through diverse fluid simulation experiments, the proposed GPU-parallelized strategy achieves a nearly 50× speedup ratio compared to the conventional CPU-FLIP method. Additionally, in the P2G stage, our method demonstrates over 30% performance improvement relative to the traditional GPU-based particle-thread scattering strategy, while the overall simulation efficiency gains exceeding 20%.

Keywords: FLIP method; fluid simulation; parallel computing; CUDA; spatial hashing index; thread block-level cooperation

Check for updates

Academic Editor: Wei Gao

Received: 20 November 2025
Revised: 15 January 2026
Accepted: 19 January 2026
Published: 21 January 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The development of fluid motion simulation has seen a continuous emergence and evolution of various methods. Stam [1] proposed an unconditionally stable implicit solver for incompressible fluids, using a semi-Lagrangian approach to solve the advection term—this approach later became the theoretical foundation for most Eulerian methods. Because Eulerian methods are based on regular grids, they excel at solving the macroscopic governing equations of fluids and handling phenomena such as diffusion and advection. The Fluid Implicit Particle (FLIP) method, proposed by Brackbill and Ruppel in 1986 [2], has gradually become one of the key approaches in fluid simulation due to its unique advantages. The FLIP method skillfully combines the strengths of both the Lagrangian method and Eulerian method, thereby enabling high-resolution, detailed fluid simulations. Zhu and Bridson [3] introduced the FLIP method into fluid simulation in computer graphics.

Concurrently, Graphics Processing Unit (GPU) technology has evolved rapidly. Originally designed for graphics rendering, GPUs offer substantial parallel computing capabilities and a highly parallel thread-level architecture [4]. To support parallel computation, they are equipped with high-bandwidth memory systems, which enable fast data read/write

operations to meet the computational needs of numerous processing cores operating concurrently [5]. Compared to CPUs, GPUs include numerous computational cores capable of processing thousands or even tens of thousands of threads concurrently, offering significant advantages in handling large-scale parallel computing tasks [6]. In fluid simulation acceleration, Harris [7] implemented Eulerian fluid simulations on GPUs using the iterative Jacobi method. Chen et al. [8] proposed a GPU-based fixed-point method to accelerate Jacobi iteration for real-time painting simulation. Xu Feng [9] implemented a parallel SPH algorithm on the GPU many-core architecture while optimizing the parallel algorithm, achieving a computational efficiency improvement of over 20 times. Jin Shanqin et al. [10] combined a particle-pair-based parallel computing method with an improved SPH approach, achieving a speedup ratio exceeding 10. Yang Zhiguo et al. [11] applied GPU algorithms to two-dimensional dam-break simulations, also obtaining acceleration effects by orders of magnitude. RENHE et al. [12] utilized tensor field reconstruction in the projection step, implementing directional modulation of fluids through a GPU parallel strategy. Guan Yanmin's team [13] established particle reordering technology and proposed an improved GPU acceleration strategy, significantly enhancing the efficiency of the SPH method.

In recent years, GPU-based parallel acceleration of the FLIP method has achieved notable breakthroughs through algorithmic restructuring and hardware–software co-design. Kui Wu [14] proposed the GVDB sparse voxel architecture, enabling dynamic grid construction, efficient parallel rasterization of particles to voxels, and a matrix-free conjugate gradient solver for sparse grids. However, its reliance on the GVDB data structure for sparse fluid simulation limits its generalizability and portability. Xin Jianjian et al. [15] developed an efficient parallel solver using the virtual grid method, making full use of shared memory and efficiently solving the Poisson equation. Nonetheless, parallelization was only achieved for the pressure Poisson equation in fluid simulation, without fully parallelizing the entire simulation process. In the field of distributed computing, Ma [16] supported multi-phase flow simulation at the hundreds-of-millions grid scale through fine-grained parallelism and communication-hiding strategies. Mao [17] integrated machine learning to accelerate fluid simulation, achieving an efficiency improvement of two orders of magnitude in reactive flows.

Despite these advancements in GPU-accelerated FLIP methods, a fundamental performance bottleneck persists in the critical particle-to-grid mapping (P2G) stage when using the intuitive scattering strategy for parallelization. In this strategy, each particle thread independently scatters its attributes to the surrounding grid nodes. While logically straightforward, it results in severe write conflict because multiple particles (handled by concurrent threads) frequently influence the same grid node. To ensure computational correctness within the CUDA programming model [18], this necessitates the use of atomic operations, which serializes memory accesses.

To directly address this atomic operations bottleneck, we propose a novel GPU parallelization strategy that fundamentally restructures the P2G workflow. Instead of the particle-centric scattering paradigm, we introduce a grid-centric approach based on two key innovations: (1) a spatial hashing index that efficiently organizes particles for coherent neighborhood queries, and (2) a thread block-level cooperation mechanism assigning entire grid cells to thread blocks for conflict-free parallel accumulation. This strategy is designed to completely eliminate atomic operations during the P2G interpolation. By migrating the FLIP method to the GPU using this optimized framework, we aim not only to leverage massive parallelism but also to achieve higher computational efficiency.

2. FLIP Algorithm in Fluid Simulation

2.1. Related Work

The FLIP method integrates Lagrangian and Eulerian perspectives. The Lagrangian method tracks individual particles, enabling accurate modeling of material transport and fluid deformation. The Eulerian method discretizes the computational domain into a regular grid, where the macroscopic fluid motion is described by solving the governing equations on grid nodes. Boyd et al. [19] proposed the Multi-FLIP algorithm, which extended the classical FLIP framework to handle multi-phase flows innovatively. While that work expanded FLIP's applicability to complex interactions, it focused on algorithmic correctness and physical fidelity rather than optimizing computational performance for real-time execution on parallel architectures such as GPUs. To improve the flexibility of simulations, Yang et al. [20] introduced an adaptive FLIP method that dynamically adjusts simulation parameters. Although this adaptability improves efficiency in some scenarios, the method does not restructure the parallel P2G step. Addressing numerical dissipation and stability, Ferstl et al. [21] achieved a breakthrough in memory efficiency and accuracy with the NarrowBand FLIP method, which confines computation near the fluid surface. However, its dynamic and irregular data structures are designed for CPU execution and are not suited for maximizing throughput on massively parallel GPUs.

In summary, prior research has significantly improved the physical realism, adaptability, and numerical stability of the FLIP method. However, most prior works extended FLIP at the algorithmic level and did not address the core performance bottleneck for large-scale GPU parallelism: severe thread serialization caused by atomic operations during the parallel P2G step. Particularly, when the simulation scale exceeds millions of particles, this bottleneck leads to a drastic degradation in parallel efficiency (see experiments in Section 4.2). Consequently, the focus of this work differs from prior research: rather than proposing a new physical variant of FLIP, we concentrate on reconstructing the data-parallel paradigm of the parallel P2G step on a GPU. Inspired by spatial indexing concepts from neighborhood search, we aim to design a parallel P2G strategy that completely avoids atomic operations and is tailored to the GPU memory hierarchy, thereby addressing the real-time challenge of large-scale, high-density fluid simulation. This forms the direct motivation for the proposed GPU acceleration method based on spatial hashing and thread block-level cooperation.

2.2. Algorithm Steps of FLIP

(1) Particle Advection

In this step, particle positions are predicted for the next time step based on the current velocity field. For higher accuracy, the fourth-order Runge–Kutta integration method [22] (Equation (1)) is used for calculation:

$$\begin{aligned} u_{temp1} &= u_g(x_p) \\ u_{temp2} &= u_g(x_p + \tfrac{1}{2}\Delta t u_{temp1}) \\ u_{temp3} &= u_g(x_p + \tfrac{1}{2}\Delta t u_{temp2}) \\ u_{temp4} &= u_g(x_p + \Delta t u_{temp3}) \\ x_p &= x_p + \frac{\Delta t(u_{temp1}+2u_{temp2}+2u_{temp3}+u_{temp4})}{6} \end{aligned} \tag{1}$$

where $u_g(x_p)$ is the velocity field function returning the velocity at position x_p, and u_{temp1}, u_{temp2}, u_{temp3},u_{temp4} are intermediate slope vectors used in the fourth-order Runge–Kutta (RK4) calculation.

(2) Particle-to-grid mapping (P2G)

In the P2G step, based on the positions x_p^n and velocities u_p of particles surrounding each grid cell, the interpolation weight coefficients ω_p of nearby particles are computed using a unified interpolation kernel function (such as trilinear interpolation). The particle velocities are then weighted and accumulated as grid mass values $M_g += \omega_p \cdot m_p$, while the sum of interpolation weights (the grid cumulative momentum) $Q_g += \omega_p \cdot m_p \cdot u_p$ is also accumulated. The grid velocity $u_g = \frac{Q_g}{M_g}$ is calculated as the ratio of the accumulated momentum to the accumulated mass. For empty grid cells where no particles are present nearby, the velocity is set to zero.

(3) Apply external force

Following the P2G step, external body forces (such as gravity) are applied to the grid velocity field u_g^*. Ignoring the pressure gradient, through explicit time integration, using forward Euler integration, the intermediate grid velocity field u_g^* is computed as the following Formula (2):

$$u_g^* = u_g + a_{external}\Delta t \tag{2}$$

where $a_{external}$ is the acceleration produced by external forces and Δt is the time step.

It is important to note that this intermediate velocity field u_g^* does not yet satisfy the fluid's incompressibility condition (i.e., zero divergence), nor does it fully comply with boundary conditions (e.g., no-slip). Therefore, this field is merely a physically incomplete transitional state. It must subsequently undergo boundary treatment, followed by a pressure projection step to correct it, in order to achieve fluid incompressibility.

(4) Apply boundary conditions

For a fluid region, boundary conditions must be specified, which define the fluid's behavior on the physical boundaries of the fluid region. When simulating liquids, there are two types of boundary conditions: the solid boundaries and the free-surface boundaries. At a solid boundary, the condition is $u_g \cdot \vec{n} = 0$, where $\vec{n}$ is the normal vector of the boundary surface. This condition ensures that the fluid cannot flow into a boundary.

(5) Grid solution and velocity field update

The core objective of the grid solver is to satisfy the incompressibility condition. The algorithm needs to find a pressure field p by constructing the pressure Poisson equation (PPE) based on an intermediate velocity field, as shown in Equation (3):

$$\nabla^2 p = \frac{\rho}{\Delta t}\nabla \cdot u_g^* \tag{3}$$

Combined with the free-surface ($p = 0$) and solid boundary (zero normal velocity gradient) conditions, the pressure field p is obtained by solving a system of linear equations, the intermediate velocity is implicitly corrected by the pressure gradient ∇p, and the grid velocity is updated as shown in Equation (4):

$$u_g^{n+1} = u_g^* - \frac{\Delta t}{\rho}\nabla p \tag{4}$$

where u_g^{n+1} is the corrected grid velocity field at time step $n+1$.

(6) Grid-to-particle mapping (G2P)

The updated grid cell velocities are interpolated back to the particles to update the particle velocities. Using a trilinear interpolation function $U(u_g, x_p)$, the new grid cell velocity $U(u_g^{n+1}, x_p^n)$ and the old grid cell velocity $U(u_g^n, x_p^n)$ at the particle position are obtained. The particle velocity field u_p^n is then updated using the FLIP-PIC hybrid method, as shown in Equation (5):

$$u_p^{n+1} = \alpha_{FLIP}\left(u_p^n + U(u^{n+1}, x_p^n) - U(u^n, x_p^n)\right) + (1 - \alpha_{FLIP})U(u^{n+1}, x_p^n) \tag{5}$$

where α_{FLIP} is the FLIP coefficient, u_p^{n+1} is the updated particle velocity field at the n + 1-th time step, u_p^n is the updated particle velocity field at the n-th time step, and x_p^n is the particle position at the n-th time step.

Following the above FLIP method steps, we built a GPU-based FLIP fluid simulation algorithm. The detailed procedure is outlined in Algorithm 1.

Algorithm 1: FLIP Simulation Procedure on the GPU

For each timestep t **do**
  For each particle p **do**
    $x_p \leftarrow particles_advect(x_p)$;
  End
  Perform spatial indexing; Section 3.2.1
  For each grid cell **in parallel do**
    $u_g \leftarrow particles_to_grid(u_P, x_p)$; Section 3.2.2
  End
  For each grid cell g at location (x, y, z) **in parallel do**
    Apply external force: $u_g^* = u_g + a_{external}\Delta t$;
  End
  Apply Boundary conditions;
  // Grid solution and velocity field update
  Solve Poisson pressure equation to obtain pressure p;
  For each grid cell g **in parallel do**
    Compute the new velocity field at the cell: $u_g = u_g^* - \Delta t \frac{\nabla p}{\rho}$;
  End
  For each particle p **do**
    $u_p \leftarrow grid_to_particles(u, x_p)$;
  End
End

3. GPU-Based Parallel Computing

In GPU-accelerated FLIP, constructing an efficient parallel algorithm framework is central to overcoming computational bottlenecks. During the parallelization of FLIP procedures in fluid simulation, steps including particle advection, applying external force, applying boundary conditions, and grid-to-particle (G2P) mapping are executed within the main simulation loop. A key advantage is the data independence between iterations, where data processed in one iteration does not depend on another. Therefore, parallelizing these operations by replacing sequential loops is straightforward and safe. In contrast, the step of P2G requires more sophisticated handling to achieve high parallel efficiency.

3.1. FLIP Parallel Algorithm Framework on GPU Architecture

As shown in Figure 1, our GPU-FLIP framework comprises preprocessing (flow field initialization), data transfer, the FLIP parallel algorithm kernel (including particle advection, mapping of particles to grids, applying external force, grid solution, and velocity field update), and free memory.

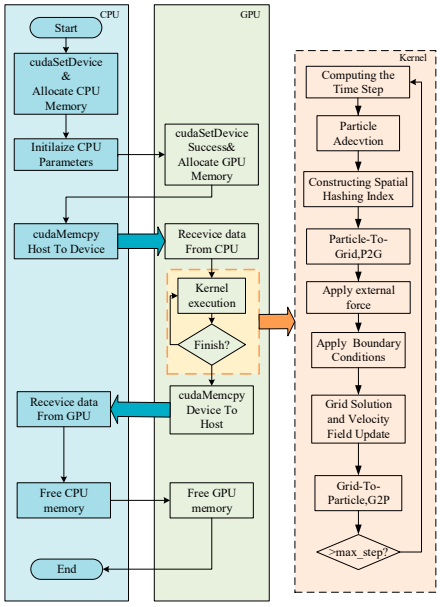


Figure 1. FLIP method GPU parallel algorithm framework.

3.2. Parallel Optimization Strategy for Particle-to-Grid Mapping (P2G)

The particle-to-grid mapping is a critical step that transfers particle attributes to the Eulerian grids. Within GPU parallel frameworks, two principal parallelization strategies are employed: gathering and scattering.

Gathering: This approach launches one thread per grid node to collect attributes from all neighboring particles within the node's support radius. The method typically requires pre-building and maintaining a spatial data structure, enabling each grid node to efficiently track and access all nearby particles.

Scattering: This method launches one thread per particle. Each thread scatters the particle's attributes to all influenced grid nodes.

For the parallel P2G step, in the scattering strategy, multiple particle threads may update the same grid node concurrently, making write conflict a critical problem, requiring atomic operation to prevent data races and ensure correctness. However, with millions of particles, write contention for grid nodes becomes severe. Unlike readily parallelizable steps like particle advection, this results in numerous atomic operations, causing thread queuing and waiting. As shown in Section 4.2.1, atomic overhead accounts for over 40% of P2G time in large-scale scenarios, and this ratio increases with particle count, severely limiting parallel efficiency. Consequently, atomic operations become the critical bottleneck to achieving higher computational performance. Therefore, optimizing P2G parallelization to eliminate or minimize atomic operation overhead is essential.

3.2.1. Spatial Hashing Index

In the FLIP method, the particle-to-grid mapping step requires searching for particles within the support radius (Δx) of each grid cell and associating them with corresponding grid cells to complete attribute interpolation. This process shares similarities with the neighbor particle search in purely particle-based methods like SPH (smoothed particle hydrodynamics) for fluid simulation, where establishing neighbor relationships involves identifying adjacent particles within a specified support radius (Δx) of a reference particle. Neighbor search constitutes an essential step in SPH algorithms. For parallel computation of SPH neighbor search algorithms [23], two primary methods for building neighbor lists are generally employed: the Cell Linked List (CLL) method and the Verlet List (VL) method [24]. The CLL method requires partitioning the computational domain into virtual

background cells. Based on the core idea shared by the uniform grid method used by Yang [25] in accelerating Ewald summation—that is, to transform global search problems into local queries through spatial partitioning—we propose that GPUs build spatial hashing index reorganization calculations in parallel to avoid atomic operations.

In three-dimensional space, when each cubic grid cell has a side length of Δx, locating all particles within radius Δx around each cell can be simplified by first identifying particles within individual cells. For any specific cell, it suffices to examine only its 26 immediately adjacent cells (totaling 27 cells including itself), since all particles within radius Δx are necessarily contained within these 27 cells.

The construction steps are as follows:

1. **Cell Hashing:** For each particle p at position $(p.x, p.y, p.z)$, we compute the index of the grid cell it resides in: cell.x $=$ floor$(p.x/\Delta x)$, cell.y $=$ floor$(p.y/\Delta x)$, cell.z $=$ floor$(p.z/\Delta x)$. We then compute a unique hash key for this cell using a linearizing function: hash_key$(\text{cell}.x, cell.y, cell.\text{z}) = \text{cell}.x \times \text{size}Y \times sizeZ + cell.y \times sizeZ \times cell.z$ for a grid of size $(sizeX, sizeY, sizeZ)$. This function ensures no hash collisions, as it provides a one-to-one mapping between 3D cell coordinates and a 1D key. We store this hash key, hash_key$(\text{cell}.x, cell.y, cell.\text{z})$, in an array, $CellHash$, where each element corresponds to a particle and records the hash key of the grid cell that contains the particle.
2. **Sorting:** We sort the array of particle indices based on their $CellHash$ using the function $thrust :: sort_by_key()$ from the CUDA Thrust library. This physically reorders the particle data array so that all particles belonging to the same grid cell become contiguous in memory.
3. **Building Cell Start/End Indices:** Create two index arrays, $cellStart$ and $cellEnd$, which represent the starting particle index and ending particle index of adjacent particles in each cell, respectively. After sorting, we launch a kernel with one thread per particle to identify the boundaries where the $hash_key$ changes. We fill two index arrays, $cellStart$ and $cellEnd$. For a given cell with $hash_key = h$, $cellStart[h]$ points to the first index in the sorted particle array that belongs to cell H, and $cellEnd[h]$ points to the last index. This allows constant-time lookup of all particles within any specific grid cell.

The 2-dimensional spatial hashing index construction process is shown in Figure 2.

The total overhead of this indexing process is minimal compared to the P2G computation, as shown in Section 4.2.1, and enables highly efficient, coherent memory access in the subsequent step.

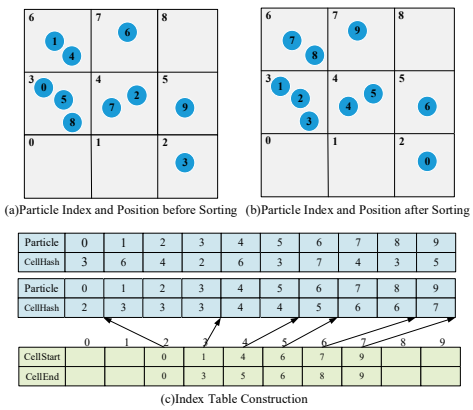


Figure 2. Basic principles of 2-dimensional spatial hashing index. Blue circles represent particles and numbers inside circles represent the unique particle indices. (labeled 0–9). Arrows illustrate the index mapping between data structures. Numbers indicate the hash indices of grid cells (0–8).

3.2.2. Thread Block-Level Cooperation Optimization Strategy

For the GPU-P2G step, we propose an optimization strategy based on thread block-level cooperation. Leveraging the efficient memory access pattern established through spatial hashing index construction, the interpolation computation tasks for each grid cell are assigned to an independent thread block. By adopting a one-to-one mapping strategy between grid cells and thread blocks, combined with intra-block thread neighborhood parallel accumulation and hierarchical reduction techniques to complete the interpolation, the method effectively addresses the load imbalance issues inherent in traditional global thread allocation strategies, thereby significantly enhancing the parallel efficiency of GPU-P2G.

The thread block-level cooperative P2G kernel is launched with a one-to-one mapping between thread blocks and grid cells. Thus, the total number of thread blocks equals the number of grid cells in the simulation domain, and each thread block contains a fixed number of threads (denoted as BLOCK_SIZE, i.e., threads per thread block). The mapping from thread block index to 3D grid coordinates is implemented as follows.

Thread Block Mapping Strategy: The address mapping employs a two-level mapping relationship, collectively forming a unique "thread block → grid cell" mapping, as follows:

Level 1: Thread block index → grid cell linear index, as shown in Equation (6).

$$cellIdx = blockIdx.x \tag{6}$$

where $blockIdx.x$ represents the thread block index.

Level 2: Grid cell linear index → 3D grid coordinates, as shown in Equation (7):

$$\begin{cases} z = cellIdx \mathrm{mod} sizeZ \\ y = \left\lfloor \frac{cellIdx}{sizeZ} \right\rfloor \mathrm{mod} sizeY \\ x = \left\lfloor \frac{cellIdx}{sizeZ * sizeY} \right\rfloor \\ x \in [0, sizeX - 1], y \in [0, sizeY - 1], z \in [0, sizeZ - 1] \end{cases} \tag{7}$$

where $sizeX$, $sizeY$, and $sizeZ$ represent the resolution of the three-dimensional grid along the (x, y, z) axes, respectively.

Each thread block is assigned to process a specific grid cell, enabling efficient spatial positioning while ensuring complete grid traversal and conflict-free access. Once a thread block is assigned to a specific grid cell (referred to as the target cell), it proceeds through the following four-phase workflow:

(1) Neighbor Cell Identification via Spatial Hashing Index: The thread block first identifies the 26 adjacent cells surrounding the target cell (27 cells, including itself). For each neighbor cell, the block computes its *hash_key* and queries the pre-built $cellStart$ and $cellEnd$ arrays to obtain the start and end indices of the particle list within that cell.

(2) Intra-Block Thread Neighborhood Parallel Accumulation: Within each thread block, all threads collaboratively traverse the combined particle list from the 27 neighboring cells. Each thread processes a distinct, equally spaced subset of particles. For each particle, it calculates the interpolation weights to the target cell and accumulates the resulting momentum and mass contributions into dedicated, per-thread locations within shared memory arrays (e.g., $M_g[threadIdx.x]$). This collaborative strategy ensures perfect load balance and transforms scattered global memory writes into efficient, on-chip data aggregation, as detailed in Algorithm 2.

(3) Hierarchical Reduction Architecture: The framework incorporates a two-level parallel reduction strategy—Warp-Level Reduction and Block-Level Reduction.

Warp-Level Reduction: Within each warp (32 threads), we use CUDA's warp shuffle instructions (*__shfl_down_sync*) to efficiently sum the contributions of the 32 threads for the target cell.

Block-Level Reduction: The partially reduced results from each warp are then further aggregated across the entire thread block using a classic tree reduction algorithm in shared memory. This step yields the final total momentum and mass contributions for the target cell.

(4) Atomic-Free Write-Back to Global Grid Arrays: The final step is to write the computed grid velocities to global memory. Since each grid node is uniquely owned by one thread block (the block responsible for the cell to which the node belongs), this write operation is inherently conflict-free and requires no atomic operations.

As illustrated in Figure 3, the proposed method fundamentally reconstructs computational tasks from a "particle-centric" to a "grid-centric" paradigm through spatial hashing index construction, completely transforming the parallel architecture of P2G. This approach entirely eliminates the massive atomic operations that occurred when multiple particle threads attempted scattered writes to the same grid nodes. By implementing thread block-level load balancing, efficient spatial hashing table queries, intra-block thread neighborhood parallel accumulation, and hierarchical reduction, the originally random, high-contention memory access pattern is transformed into a regular, low-contention intra-block computational pattern. This transformation substantially enhances both the parallel computational efficiency and overall performance of the P2G step, as shown in Algorithm 2.

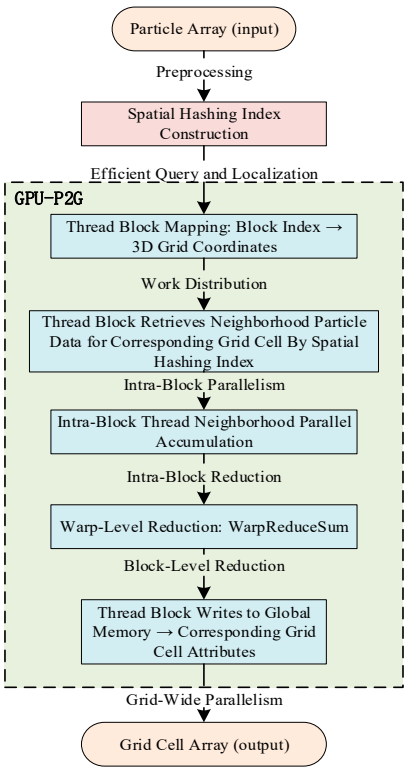


Figure 3. P2G-GPU parallel optimization strategy flowchart.

Algorithm 2: Thread-Block Cooperative P2G Kernel

Input: Sorted particle data, $cellStart$, $cellEnd$
Output: Updated grid velocity $u_g(c_x, c_y, c_z)$.
// Thread Block Mapping
$cell_pos(c_x, c_y, c_z) \leftarrow \text{Map}(blockIdx.x)$; // Equations (6) and (7)
Allocate and Initialize $M_g[BLOCK_SIZE] = 0$, $Q_g[BLOCK_SIZE] = 0$ in shared memory // $BLOCK_SIZE$ is neighboring cell's counts
// Intra-Block Thread Neighborhood Parallel Accumulation
For each grid cell of 27 neighboring cells **in parallel do**
  // compute neighboring grid cell hash index:
  $neighborcell_hash = CellHash(neighborcell_idx)$
  $start \leftarrow cellStart[neighborcell_hash], end \leftarrow cellEnd[neighborcell_hash]$
  If $start > 0$:
    For particle index $idx = start + thredIdx.x; idx < end; idx += BLOCK_SIZE$ **do**
      Load particle p at index idx
      $\omega_p = InterpWeight(p, pos)$
      $M_g[thredIdx.x] += \omega_p \cdot m_p$
      $Q_g[thredIdx.x] += \omega_p \cdot m_p \cdot u_p$
    End
  End
End
__syncthreads() // Ensure all particle processing is complete
// Warp-level intra-block reduction (using warp shuffle)
$M_g[warp_leader] = warpReduceSum(M_g[:])$
$Q_g[warp_leader] = warpReduceSum(Q_g[:])$
// Block-Level Reduction
If $thredIdx.x < num_warps$ **then**
  $final_M_g = blockReduceSum(M_g[threadIdx.x])$
  $final_Q_g = blockReduceSum(Q_g[threadIdx.x])$
End
__syncthreads()
// Write back results (no atomic operations)
Write $\frac{final_Q_g}{final_M_g}$ to update grid velocity $u_g(c_x, c_y, c_z)$

4. Experimental Results and Analysis of GPU Parallel Acceleration

4.1. Operating Environment

To minimize potential interference from cold-start effects and background processes while ensuring data stability, we recorded the average execution time over 200 simulation frames in multiple runs. Three configurations were compared: the CPU-FLIP algorithm (labeled CPU), the GPU-FLIP algorithm using particle-thread scattering for P2G (labeled GPU_1), and the GPU-FLIP algorithm employing spatial hashing and thread-block cooperation for P2G (labeled GPU_2). The experiments were compiled in Microsoft Visual Studio 2022 with CUDA 12.6. Hardware specifications of the testing platform are summarized in Table 1.

Table 1. CPU and GPU performance parameters.

Device	Model	Number of Cores	Memory/Graphics Memory (GB)
CPU	AMD Ryzen 77,840 H with Radeon 780 M Graphics	8	16
GPU	NVIDIA GeForce RTX 4060 Laptop GPU	3072	8

4.2. Performance Analysis

Experiments were conducted using dam-break simulations and droplet falling simulations with grid resolutions of 128^3, 160^3, 190^3, and 224^3. The CPU experiments utilized the classical fluid simulation framework MantaFlow, while the GPU-accelerated FLIP methods were implemented using the CUDA framework. To ensure experimental fairness, all GPU parallel simulations were based on the same code foundation, differing only in the strategies employed for the P2G step.

4.2.1. Spatial Hashing Index Construction Overhead Analysis

To quantitatively assess the efficiency gained by eliminating atomic operations through spatial hashing index construction, this experiment recorded and analyzed the average computation time required for building the spatial hashing index during dam-break simulations.

As shown in Table 2, although the construction of the spatial hashing index introduces additional computational overhead within the P2G step, it does not significantly impact the overall simulation performance or reduce computational efficiency. The minor additional cost is substantially outweighed by the considerable improvement in the overall computational efficiency of the P2G step.

Table 2. Average time taken to build a spatial hashing index during a dam-break simulation (ms).

Particle	Grid	Build Hash Array	Sort Particles	Create Index
3.5 M	128^3	0.73	5.12	0.20
7 M	160^3	1.34	10.37	0.36
12 M	192^3	2.23	16.15	0.59
20 M	224^3	3.45	24.82	0.89

4.2.2. GPU-P2G Runtime Comparison

We measured and analyzed the average execution time of the P2G step in GPU-FLIP during dam-break simulations across varying grid resolutions and particle counts. This analysis compares the particle-thread scattering strategy with two spatial hashing index-based approaches—the global thread allocation strategy and the thread block-level cooperation optimization strategy—in the context of parallel fluid simulation, thus effectively evaluating their respective performance in mitigating the key bottleneck of atomic operations (where 'Improv.' is the abbreviation for 'Improvement').

As shown in Table 3, in the dam-break simulation with a 192^3 grid and 12 million particles, the particle-thread scattering strategy required 93.93 ms, while the spatial hashing index-based global thread strategy and the thread block-level cooperation optimization strategy took 68.33 ms and 49.46 ms, respectively. These execution times correspond to significant performance improvements of approximately 27% and 47%, demonstrating substantial enhancement in the computational efficiency of the P2G step within the GPU-FLIP fluid simulation.

Table 3. Comparison of average time consumption of P2G steps in dam-break simulation.

Particle	Grid	Particle Thread Scattering Strategy/ms	Global Thread Strategy/ms	Improv.	Thread Block-Level Cooperation Optimization Strategy/ms	Improv.
3.5 M	128^3	23.55	18.30	22%	15.23	35%
7 M	160^3	46.25	37.39	19%	29.05	37%
12 M	192^3	93.93	68.33	27%	49.46	47%
20 M	224^3	171.28	111.74	35%	78.28	54%

To further demonstrate the performance improvement of the thread block-level cooperation optimization strategy, Table 4 presents results from a dam-break simulation with a 224^3 grid and 20 million particles. The traditional global thread strategy required 111.74 ms for the P2G step, while the thread block-level cooperation optimization strategy completed it in 78.28 ms, achieving a performance improvement of approximately 30%. Furthermore, as both grid resolution and particle count increase, the advantage of the thread block-level cooperation optimization strategy becomes increasingly pronounced.

Table 4. Comparison of average time consumption of P2G step (based on spatial hashing index) thread strategies in dam-break simulation.

Particle	Grid	Global Thread Strategy/ms	Thread Block-Level Cooperation Optimization Strategy/ms	Improv.
3.5 M	128^3	18.30	15.23	17%
7 M	160^3	37.39	29.05	22%
12 M	192^3	68.33	49.46	28%
20 M	224^3	111.74	78.28	30%

4.2.3. Thread Block-Level Mapping Optimization Strategy: Runtime Comparison

To better analyze the computational efficiency improvement of the proposed optimization strategy on the entire fluid simulation, this experiment recorded a detailed time for each major step—P2G, G2P, (Particle) advection, and pressure solve—during a dam-break simulation with a 192^3 grid and 12 million particles.

As shown in Table 5, the time breakdown reveals that in the particle-thread scattering strategy (GPU_1), the P2G step is the most time-consuming component due to massive atomic operations causing warp serialization. Notably, both (particle) advection and G2P steps in GPU_1 require more time than in GPU_2. A plausible explanation is that the extremely high atomic operation density and random memory access patterns during P2G degrade the state of the GPU's memory subsystem, and this degraded state persists into the subsequent G2P and particle advection steps of the next cycle. The pressure solve step remains relatively unaffected, as it primarily operates on grid velocity and pressure fields with data isolation and regular computational patterns.

By adopting the spatial hashing index construction and thread block-level cooperation optimization strategy, the method effectively eliminates the adverse effects of atomic operations, reduces the time proportion of P2G in GPU-FLIP fluid simulation, and consequently, enhances the overall computational efficiency.

Table 5. Time-consuming subdivision comparison of various steps in dam-break simulation.

		GPU_1/ms	GPU_2/ms
Grid: 192^3 Particle: 12 M	Advection	11.09	7.13
	P2G	93.93	49.88
	Pressure Solve	101.61	101.44
	G2P	8.52	6.4
	Other	3.19	1.17
	Total Time	218.32	166.02

4.2.4. Overall Computational Efficiency Comparison

To eliminate potential interference from cold-start effects and background processes while ensuring data stability, this experiment recorded the average execution time over 200 simulation frames across multiple runs. The average per-frame execution time was subsequently calculated for each configuration, enabling the derivation of speedup ratios for both GPU-accelerated implementations relative to the CPU simulation. These ratios provide the foundation for a robust comparison of overall computational efficiency.

Table 6 presents a computational efficiency comparison of three methods in dam-break experiments across four grid resolutions. The results demonstrate that both GPU-accelerated FLIP methods achieve significant speedup compared to the CPU-FLIP approach. At a grid resolution of 160^3, the particle-thread strategy-based GPU method (GPU_1) achieves a speedup ratio of 33.3×, while the proposed spatial hashing index and thread block-level cooperation strategy-based GPU method (GPU_2) reaches 41.5, representing an improvement of approximately 25%.

Table 6. Comparison of computational efficiency for dam-break simulations at different grid resolutions.

Grid	CPU/ms	GPU_1/ms	Speedup Ratio_1	GPU_2/ms	Speedup Ratio_2
128^3	1887.10	45.77	41.2	35.60	53.0
160^3	3842.53	115.31	33.3	92.60	41.5
192^3	7147.20	218.32	32.7	166.02	43.1
224^3	12,364.20	372.02	33.2	261.03	47.4

Given that dam-break experiments involve complex, large-deformation simulations with dense fluids, when the grid resolution increases to 224^3, the atomic operations generated during interpolation in the GPU_1 method rise dramatically, causing its speedup ratio to decline to 33.2×. In contrast, the GPU_2 method, utilizing spatial hashing index construction, demonstrates continued performance improvement as the grid scale expands, with its speedup ratio rising to 47.4×—an enhancement of approximately 42% compared to GPU_1 at this resolution.

Table 7 presents the computational efficiency comparison of three methods in droplet falling experiments across four grid resolutions. Unlike dam-break simulations that represent macroscopic large-deformation scenarios, droplet falling experiments focus more on microscopic, small-scale, sparse fluid dynamics. The results demonstrate that both GPU-parallelized FLIP methods substantially enhance the overall computational efficiency. At a 160^3 grid resolution, the GPU_1 method achieves a speedup ratio of 37×, while the GPU_2 method reaches 44.8×, representing an improvement of approximately 21%. When the grid resolution increases to 224^3, the speedup ratio of GPU_1 rises to 43.2×, and

GPU_2 further improves to 53.1×, maintaining a computational efficiency enhancement of approximately 23%.

Table 7. Comparison of computational efficiency for water droplet falling simulations at different grid resolutions.

Cell	CPU/ms	GPU_1/ms	Speedup Ratio_1	GPU_2/ms	Speedup Ratio_2
128^3	1733.55	40.17	43.2	31.82	54.5
160^3	3846.51	103.93	37.0	85.91	44.8
192^3	7308.04	185.74	39.4	153.60	47.6
224^3	12,820.96	297.08	43.2	241.65	53.1

In summary, although the particle-thread scattering strategy (GPU_1) achieves notable acceleration in small-scale, sparse fluid simulations, its performance markedly deteriorates in complex, high-density fluid scenarios. In contrast, the proposed spatial hashing index and thread block-level cooperation strategy-based GPU-FLIP algorithm (GPU_2) maintains excellent acceleration performance even in complex, dense fluid simulations, demonstrating superior robustness and scalability.

Analysis of the GPU methods in Tables 6 and 7 reveals particularly large speedup ratios at the 128^3 grid resolution. A potential explanation is the existence of a performance inflection point between 128^3 and 160^3 resolutions, where GPU hardware resources (memory bandwidth/cache) approach but do not fully saturate, creating a suboptimal blocked state. Further increasing the grid resolution pushes the hardware into full saturation, enabling computing units to maintain continuous peak utilization.

4.3. Simulation Results

To validate the visual quality of the GPU-accelerated FLIP method, this experiment demonstrated the fluid simulation effects of three different approaches at a grid resolution of 160^3. All fluid simulations in this experiment were rendered using Blender to generate the final visual results.

As shown in Figure 4, comparative analysis of the dam-break simulation results demonstrates that the proposed parallel GPU-FLIP algorithm maintains the capability to simulate complex fluid phenomena with large deformations and high kinetic energy while preserving high computational efficiency. The overall visual quality and physical authenticity effectively meet the application requirements for sophisticated fluid simulation, achieving an optimal balance between computational performance and simulation fidelity.

As shown in Figure 5, comparative analysis of the droplet falling simulation results demonstrates that the proposed GPU-FLIP method, when applied to microscopic-scale simulations of small-scale sparse fluids, achieves significant computational efficiency improvements while still preserving high-quality simulation outcomes—including detailed free-surface features and physically plausible collision interactions.

It is important to note in advance that the differences in flow field morphology observed between the CPU and GPU implementations in Figures 4 and 5 are reasonable outcomes arising from distinct engineering implementations under the same FLIP algorithmic paradigm.

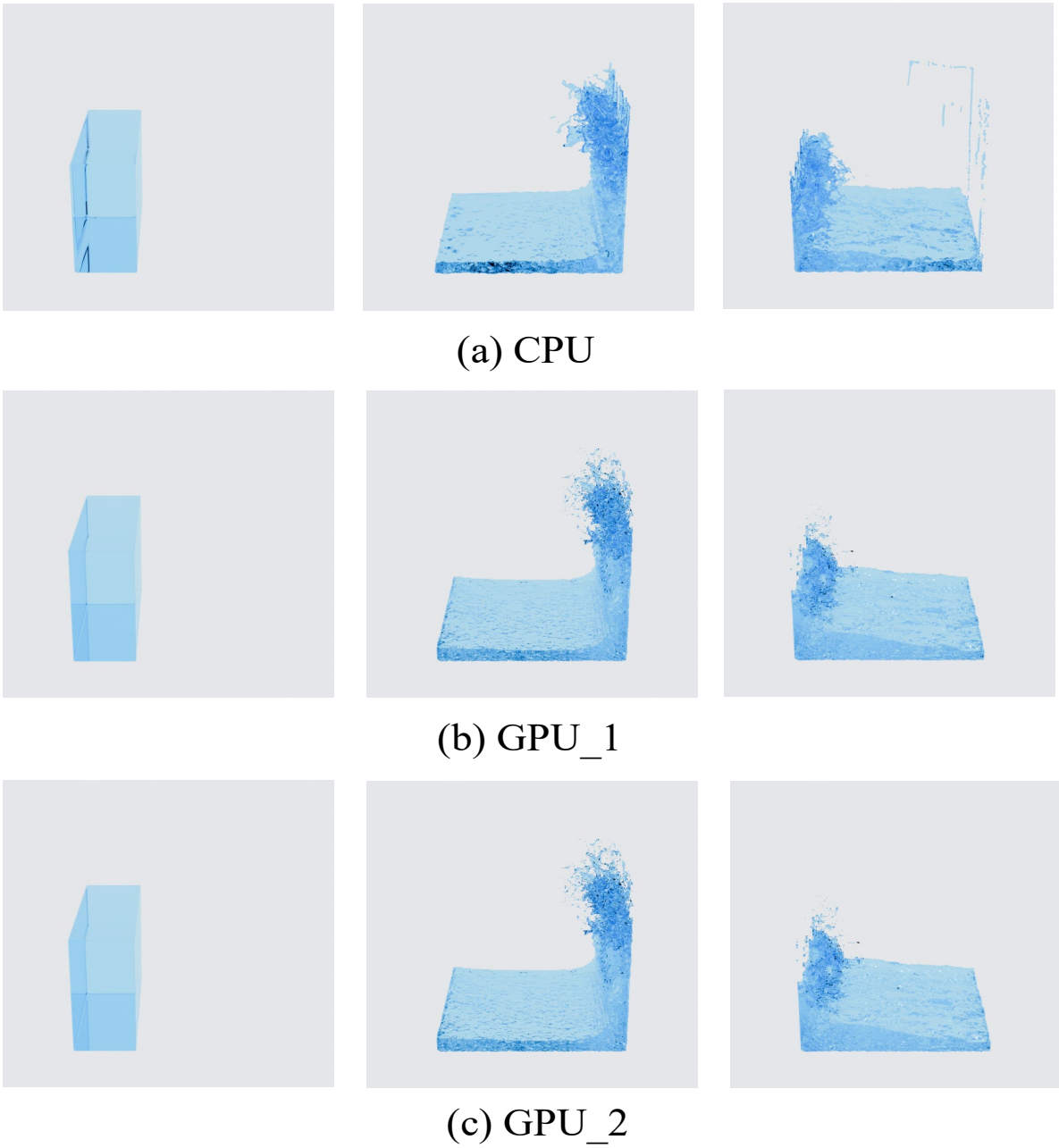


Figure 4. Dam-break simulation results.

As a hybrid particle-in-cell method, FLIP defines the core coupling process: projecting particle velocities onto a grid, solving the incompressible Navier–Stokes equations on that grid, and transferring the updated velocities back to the particles. Crucially, the algorithm inherently permits flexibility in the engineering specifics of each step (e.g., interpolation kernels, linear solver types, boundary condition implementation, and parallelization strategies). Therefore, different valid implementation paths can lead to numerically divergent—yet physically consistent—flow realizations.

We emphasize that the core innovation of this study is the optimization of atomic operations in the GPU-FLIP P2G step. All performance comparisons and analyses between our optimized method and the traditional GPU scattering method are performed within the same codebase and numerical framework. Consequently, the flow field results presented for these comparative evaluations maintain a high degree of consistency, which robustly ensures the validity of our optimization scheme. The CPU implementation is employed

solely as a performance benchmark, and its numerical differences from our GPU implementation do not affect the conclusions regarding the efficiency gains achieved by our proposed optimization.

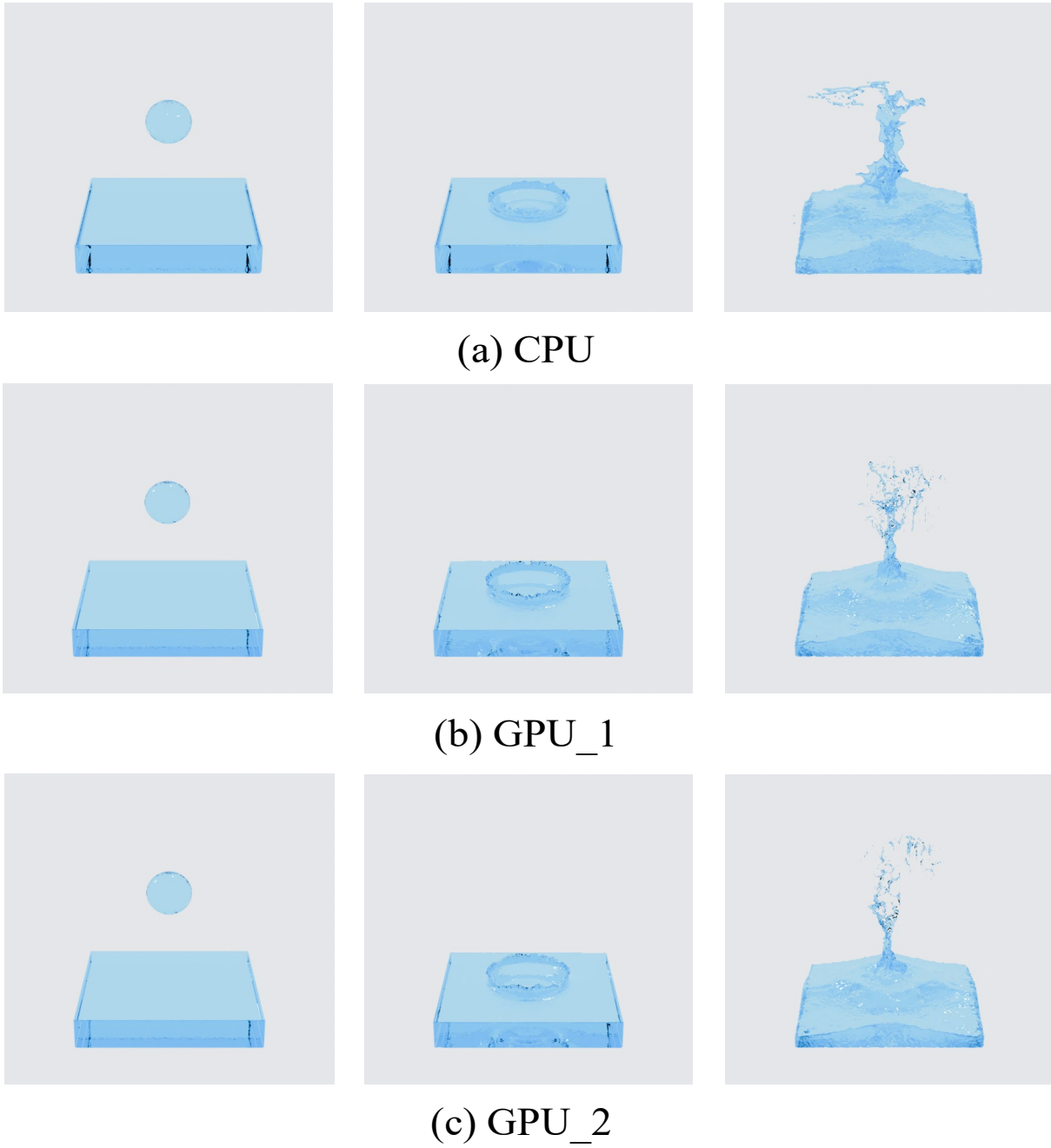
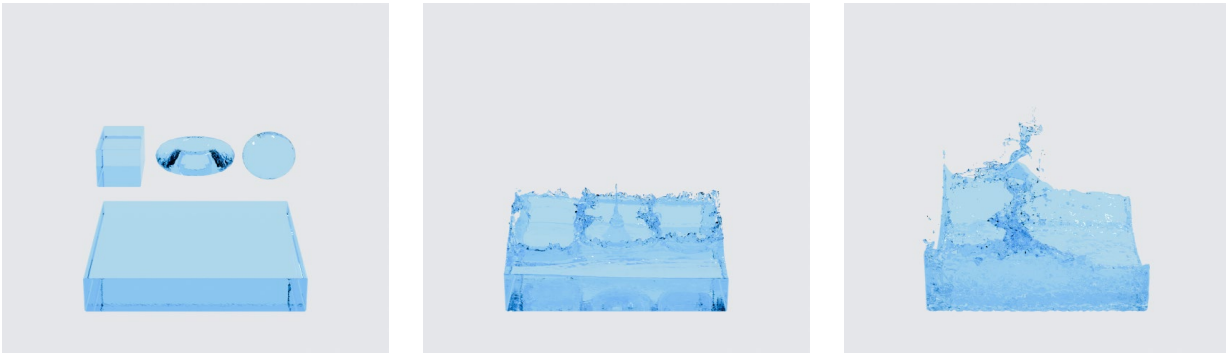


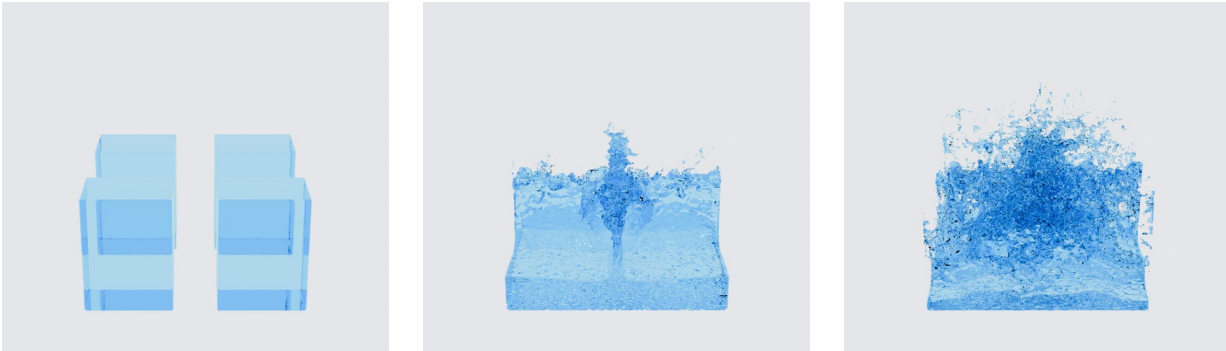
Figure 5. Droplet falling simulation results.

To further validate the method's generalizability and simulation quality, we conducted comprehensive experiments across diverse scenarios using the proposed approach (as shown in Figure 6): The experimental results demonstrate that our method consistently exhibits excellent simulation performance across all tested scenarios, thoroughly verifying its capability to reliably generate high-quality fluid simulations in complex interaction and multi-source flow environments—including accurate representation of free-surface details, maintenance of deformation continuity, and physically authentic collision interactions.



(a)free-falling fluids with multiple geometric shapes

(b) fluid interaction with the Stanford Bunny model



(c) complex multi-directional dam-break collisions

Figure 6. Multiple simulation scenarios: (**a**) free-falling fluid with multiple geometric shapes; (**b**) fluid interaction with the Stanford Bunny model; and (**c**) complex multi-directional dam-break collisions.

5. Conclusions and Future Work

Our multi-scale fluid experiments demonstrate that GPU-parallel acceleration significantly improves the FLIP method's computational efficiency while preserving its validity, yielding high-quality simulation results. Our results also show that as grid and particle scales increase, extensive atomic operations in the traditional scattering strategy cause a nonlinear decay in computational efficiency. In large-scale scenarios, atomic overhead becomes dominant, forming a critical bottleneck for further performance gains. To address this challenge, we propose a GPU optimization strategy based on spatial hashing index and thread block-level cooperation. By reconstructing the computational workflow and thread scheduling logic of the P2G step, the method completely circumvents redundant atomic operations during the P2G step, further unlocking the GPU's parallel computing

potential and effectively accelerating the fluid solving process. As demonstrated by data in Tables 3 and 6, this optimization strategy achieves a 50% performance improvement in the P2G step for dam-break simulation with 20 million particles and a 224^3 grid, while the overall computational efficiency reaches a 47.3× speedup ratio compared to CPU-FLIP, representing over 20% performance gain compared to the traditional scattering strategy. This provides an efficient and feasible technical pathway for large-scale, high-resolution fluid simulation.

The proposed strategy significantly enhances parallel interpolation performance in grid cells with high particle density by precisely locating neighboring particle information and incorporating a thread block-level cooperation mechanism. However, when adjacent grid cells exhibit significant disparities in particle density distribution, it leads to workload imbalance within the thread block—some threads become fully occupied, while others remain idle, thereby constraining further improvement in overall parallel efficiency. Future work will focus on this challenge through an in-depth investigation of optimized thread scheduling strategies to further exploit GPU parallel computing potential.

Author Contributions: Conceptualization, C.Z. and H.L.; validation, H.L.; writing—original draft preparation, H.L.; writing—review and editing, C.Z. and H.L.; supervision, C.Z.; All authors have read and agreed to the published version of the manuscript.

Funding: National Natural Science Foundation of China (No. 62162027); Humanities and Social Sciences Research Project of Jiangxi Universities (JC24205); Innovation and Entrepreneurship Education Research Project of East China Jiaotong University (24hjct18). College Student Innovation and Entrepreneurship Training Program (202410404014).

Data Availability Statement: The experimental portion of this paper contains simulation data analyzed during this study. The (CPU) experiments are run using the mantaflow engine. [https://github.com/lhluohuilh/hash_GPU-FLIP.git] (accessed on 14 November 2025) provides part of the simulation results and code files during the current study. The data generated during the current study are not publicly available due to key parts of this code affecting subsequent research, but are available from the corresponding author on reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Stam, J. Stable fluids. *ACM Trans. Graph.* **2001**, *1999*, 121–128.
2. Brackbill, J.U.; Kothe, D.B.; Ruppel, H.M. Flip: A low-dissipation, particle-in-cell method for fluid flow. *Comput. Phys. Commun.* **1988**, *48*, 25–38. [CrossRef]
3. Zhu, Y.; Bridson, R. Animating sand as a fluid. *ACM Trans. Graph.* **2005**, *24*, 965972. [CrossRef]
4. Kirk, D.B.; Hwu, W.-M.W. *Programming Massively Parallel Processors: A Hands—On Approach*; Morgan Kaufmann: Burlington, MA, USA, 2016.
5. Samadi, M.; Qureshi, M.K.; Balasubramonian, R. Memory bandwidth characterization of graphics processing units. In Proceedings of the 2010 37th Annual International Symposium on Computer Architecture (ISCA), Saint-Malo, France, 19–23 June 2010; IEEE: Piscataway, NJ, USA, 2010; pp. 160–171.
6. Nickolls, J.; Buck, I.; Garland, M.; Skadron, K. Scalable parallel programming with CUDA. *Queue* **2008**, *6*, 40–53. [CrossRef]
7. Harris, M.J. Fast fluid dynamics simulation on the GPU. In Proceedings of the ACM SIGGRAPH 2005 Courses, Los Angeles, CA, USA, 31 July–4 August 2005; ACM: New York, NY, USA.
8. Chen, Z.; Kim, B.; Ito, D.; Wang, H. Wetbrush: GPU-based 3d painting simulation at the bristle level. *ACM Trans. Graph.* **2015**, *34*, 200. [CrossRef]
9. Xu, F. Research and Implementation of a Parallel SPH Algorithm Based on Many-Core Architecture. Master's Thesis, Shanghai Jiao Tong University, Shanghai, China, 2013. (In Chinese)
10. Jin, S.Q.; Zheng, X.; Duan, W.Y. Simulation of viscous flow field using an improved SPH method with GPU acceleration. *J. Harbin Eng. Univ.* **2015**, *36*, 1011–1018. (In Chinese)
11. Yang, Z.G.; Huang, X.; Zheng, X.; Duan, W.Y. Research on the application of GPU in the SPH method to simulate the dam failure problem. *J. Harbin Eng. Univ.* **2014**, *35*, 661–666. (In Chinese)

12. Parreiras, E.A.; Vieira, M.B.; Machado, A.G.; Renhe, M.C.; Giraldi, G.A. A particle-in-cell method for anisotropic fluid simulation. *Comput. Graph.* **2022**, *102*, 220–232. [CrossRef]
13. Guan, Y.M.; Yang, C.H.; Kang, Z.; Zhou, L. Application of an improved GPU acceleration strategy in smoothed particle hydrodynamics. *J. Shanghai Jiao Tong Univ.* **2023**, *57*, 981–987. (In Chinese)
14. Wu, K.; O'brien, J.F.; Fedkiw, R. Fast fluid simulations with sparse volumes on the GPU. *Comput. Graph. Forum* **2018**, *37*, 157–167. [CrossRef]
15. Xin, J.J.; Shi, F.L. A GPU-CUDA parallel virtual grid finite difference solver. *J. Hydrodyn.* **2023**, *38*, 523–527. (In Chinese)
16. Ma, K.; Jiang, M.; Liu, Z. Accelerating fully resolved simulation of particle-laden flows on heterogeneous computer architectures. *Particuology* **2023**, *81*, 25–37.
17. Mao, R.; Zhang, M.; Wang, Y.; Li, H.; Xu, J.; Dong, X.; Zhang, Y.; Chen, Z.X. An integrated framework for accelerating reactive flow simulation using GPU and machine learning models. *Proc. Combust. Inst.* **2024**, *40*, 105512. [CrossRef]
18. NVIDIA. CUDA C Programming Guide [EB/OL]. (2013-06) [2024-07-01]. Available online: https://docs.nvidia.com/cuda/cuda-c-programming-guide/ (accessed on 6 October 2025).
19. Boyd, L.; Bridson, R. Multi FLIP for energetic two-phase fluid simulation. *ACM Trans. Graph.* **2012**, *31*, 16. [CrossRef]
20. Yang, L.P.; Li, S.; Xia, Q.; Hong, Q.; Aimin, H. A novel integrated analysis-and-simulation approach for detail enhancement in FLIP fluid interaction. In Proceedings of the 21stACM Symposium on Virtual Reality Software and Technology, Beijing, China, 13 November 2015; pp. 103–112.
21. Ferstl, F.; Ando, R.; Wojtan, C.; Westermann, R.; Thuerey, N. Narrow band FLIP for liquid simulations. *Comput. Graph. Forum* **2016**, *35*, 225–232. [CrossRef]
22. Gottlieb, S.; Shu, C.W. Total variation diminishing Runge-Kutta schemes. *Math. Comput.* **1998**, *67*, 73–85. [CrossRef]
23. Zhou, P.; Jin, A.F. Application of GPU-based neighbour search method in SPH algorithm for sand-laden wind flow. *Comput. Appl. Softw.* **2025**, *42*, 221–226. (In Chinese)
24. Winkler, D.; Rezavand, M.; Rauch, W. Neighbour lists for smoothed particle hydrodynamics on GPUs. *Comput. Phys. Commun.* **2018**, *225*, 140–148. [CrossRef]
25. Yang, S.C.; Wang, Y.L.; Jiao, G.S.; Qian, H.J.; Lu, Z.Y. Accelerating electrostatic interaction calculations with graphical processing units based on new developments of Ewald method using non-uniform fast Fourier transform. *J. Comput. Chem.* **2016**, *37*, 378–387. [CrossRef] [PubMed]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.